

Thiết kế bộ sinh test case API bằng AI

HW06 · 23127195 · Mục §7 của đề bài — mức Bloom-AI G9.5 (Create)

- Đầu vào:** đặc tả API (`api_specification.md`) + đặc tả yêu cầu hệ thống (`README.md` / SRS)
- Đầu ra:** bộ test case đầy đủ, đã gắn nhãn, chạy được bằng Postman + Newman

1. Vấn đề cần giải quyết

Cách làm ngây thơ là đưa cả tài liệu đặc tả cho một mô hình ngôn ngữ kèm câu lệnh "sinh toàn bộ test case cho API này". Cách đó hỏng ở bốn điểm, và cả bốn đều quan sát được trong chính bài tập này:

#	Vấn đề	Bảng chứng quan sát được
1	Không thể kiểm toán. Không biết test case nào phủ yêu cầu nào, nên không trả lời được câu hỏi "đã phủ đủ chưa".	—
2	Phủ không đều. Mô hình sinh dày ở phần dễ (status code, trường bắt buộc) và mỏng ở phần khó (chuyển trạng thái, bất biến nghiệp vụ).	Không có case nào cho <code>ISSUED → ISSUED</code> , cho cập nhật một phần, cho vị trí dòng lỗi trong mảng
3	Kỳ vọng lấy từ hành vi thật thay vì từ đặc tả. Khi được cho xem response thật, mô hình có xu hướng chép lại nó thành "kỳ vọng".	Xem <code>AI_CRITIQUE.md</code> — mục <i>neo theo hành vi quan sát được</i>
4	Sinh thẳng ra mã JavaScript thì lỗi cú pháp và lỗi logic bất đồng bộ nằm rải rác, không rà soát nổi.	32 assertion bất đồng bộ nuốt mất lỗi — xem §5 dưới đây

Thiết kế này xử lý cả bốn bằng cách **tách phần suy luận ngôn ngữ ra khỏi phần sinh mã**.

2. Nguyên tắc thiết kế

N1 — Dùng biểu diễn trung gian (IR), không sinh thẳng ra mã

LLM chỉ sinh **JSON mô tả ý định của test case**, không sinh JavaScript. Một bộ biên dịch tĩnh định (`build_collections.py`) mới dịch IR sang Postman Collection.

Vì sao: JSON có lược đồ nên **máy kiểm tra được**; assertion được mô tả bằng *descriptor* (`{ "t": "eq", "path": "final_amount", "v": 450000 }`) nên không thể sai cú pháp; và mọi test case đều được dịch bằng **cùng một** khuôn mã, nên một lỗi trong khuôn được sửa **một lần** cho toàn bộ 144 test case. Đây chính là cách hai lỗi harness ở §5 được sửa.

N2 — Bơm quy tắc, không hỏi cảm hứng

Không hỏi "hãy nghĩ ra các test case". Thay vào đó, với mỗi tham số trích được từ đặc tả, hệ thống **áp một danh mục phân vùng bắt buộc**:

mọi tham số	→ thiếu trường · null · sai kiểu · rỗng
kiểu chuỗi	→ chuỗi rỗng · chỉ khoảng trắng · vượt độ dài tối đa · Unicode · payload SQLi/XSS
kiểu số	→ 0 · số âm · vượt Number.MAX_SAFE_INTEGER · số thực · chuỗi-số
có ràng buộc biên	→ min-1 · min · min+1 · max-1 · max · max+1 (bắt buộc, không bỏ)
có ràng buộc mẫu	→ khớp · không khớp · khớp một phần · khác hoa/thường · có khoảng trắng bao quanh
khoá ngoại	→ tồn tại · không tồn tại · 0 · số âm

LLM chịu trách nhiệm phần nó làm tốt (đọc hiểu ngôn ngữ tự nhiên → trích tham số và ràng buộc).

Danh mục chịu trách nhiệm phần LLM làm kém (**tính đầy đủ**). Đây là lý do phân vùng `min` — nơi tìm ra BUG-A2-03 — không bao giờ bị bỏ sót.

N3 — Kỳ vọng phải truy vết được về đặc tả, không về hành vi

Mỗi test case bắt buộc mang trường `expected_by_spec` — trích dẫn hoặc suy luận từ tài liệu.

Test case nào không nêu được nguồn kỳ vọng thì bị chặn ngay tại bước kiểm tra lược đồ IR.

Hệ quả: bộ sinh **không bao giờ được xem response thật** trước khi sinh xong test case.

Nếu vi phạm, mọi bug sẽ được "hợp thức hoá" thành hành vi kỳ vọng.

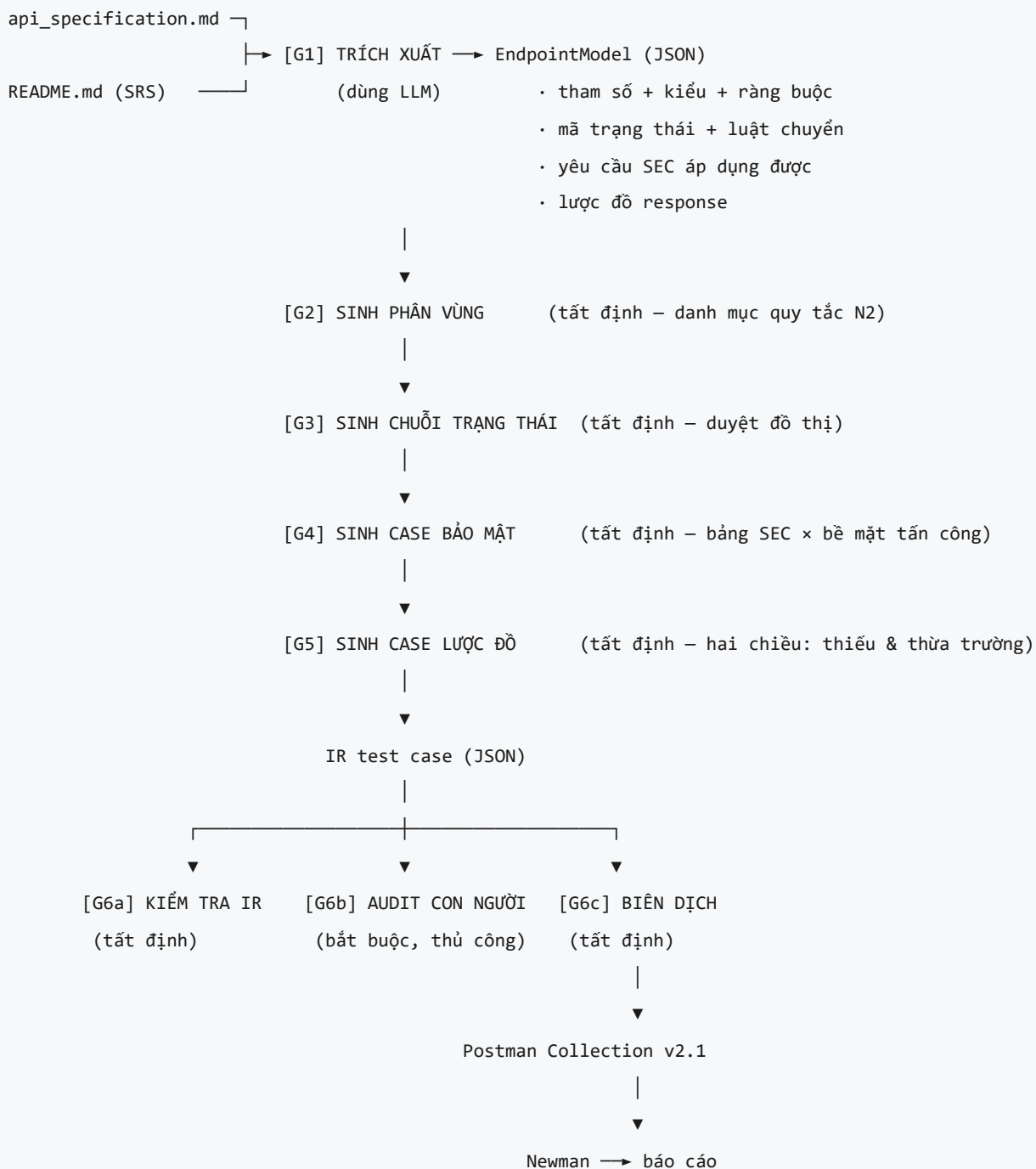
N4 — Nhãn audit là dữ liệu bắt buộc, không phải bước tùy chọn

Mỗi case mang `source` (AI / HUMAN) và `audit: {label, reason}`. Bước audit của con người không thể bị làm hình thức vì file IR **không hợp lệ** nếu thiếu các trường này.

N5 — Một nguồn sự thật duy nhất

Từ IR sinh ra: Postman collection, file Excel test case, bảng tổng hợp báo cáo, và cổng chặn hồi quy của CI. Số liệu trong báo cáo **không thể** lệch với collection, vì cả hai cùng sinh từ một chỗ.

3. Kiến trúc — sáu giai đoạn



Chỉ giai đoạn G1 dùng LLM. Năm giai đoạn còn lại là mã tất định. Đây là quyết định thiết kế trung tâm: thu hẹp bề mặt sai của mô hình xuống đúng một việc mà mô hình làm tốt là **đọc hiểu tài liệu**, rồi để mã đảm bảo tính đầy đủ và tính đúng cú pháp.

Chi tiết từng giai đoạn

G Đ	Tên	Loại	Đầu vào	Đầu ra
G1	Trích xuất mô hình endpoint	LLM	Đặc tả API + SRS	EndpointModel : danh sách tham số (tên, kiểu, bắt buộc?, ràng buộc), máy trạng thái, danh sách SEC áp dụng, lược đồ response
G2	Sinh phân vùng miền	Tất định	EndpointModel.parms	1 case cho mỗi (tham số × phân vùng) trong danh mục N2
G3	Sinh chuỗi chuyển trạng thái	Tất định	EndpointModel.state_machine	Mọi cạnh hợp lệ + mọi cạnh không hợp lệ (bao gồm cạnh tự-lập và cạnh rời trạng thái kết thúc)
G4	Sinh case bảo mật	Tất định	Bảng SEC × bề mặt tấn công	Không xác thực · token rác · token ký sai khoá · alg=none · sai vai trò · IDOR · mass assignment · SQLi trên mọi tham số chuỗi
G5	Sinh case lược đồ	Tất định	EndpointModel.response_schema	Khớp lược đồ · kiểu từng trường · không có trường thừa · Content-Type · không rõ ràng dữ liệu nhạy cảm
G6a	Kiểm tra IR	Tất định	IR	Chặn nếu: thiếu expected_by_spec , thiếu nhãn audit, trùng ID, thiếu phân vùng bắt buộc
G6b	Audit của con người	Thủ công	IR	Gắn VALID / INVALID / INCOMPLETE + lý do; sửa case sai; thêm case AI bỏ sót
G6c	Biên dịch	Tất định	IR đã audit	Postman Collection v2.1 + Excel + cổng chặn CI

4. Vì sao G6b không thể tự động hoá

Đây là ranh giới trách nhiệm của thiết kế, và bài tập này cho bằng chứng cụ thể.

34 test case do con người thêm đã tìm ra những lỗi mà không nhánh tất định nào ở G2–G5 sinh được, vì chúng đòi hỏi hiểu **hậu quả nghiệp vụ** chứ không phải cấu trúc dữ liệu:

Test case con người thêm	Lỗi tìm được	Vì sao máy không sinh được
TC-A1-028 cập nhật một phần	BUG-A1-05 mất dữ liệu âm thầm	Đặc tả liệt kê 3 trường; không chỗ nào nói " <i>client có thể chỉ gửi một trường</i> "
TC-A3-033/034 vị trí dòng lỗi	BUG-A3-02 (kết luận đúng về tính nguyên tử)	Cần hiểu rằng "chạy tuần tự rồi dừng" và "giao dịch" cho cùng kết quả khi lỗi ở dòng đầu
TC-A3-036 báo cáo phải nhất quán với CSDL	BUG-A3-02 (mức nghiêm trọng thật)	Cần hiểu người vận hành sẽ hành động dựa trên con số <code>inserted</code>
TC-A2-036 bất biến <code>0 ≤ discount ≤ total</code>	BUG-A2-02	Kiểm theo bất biến bắt được mọi cách tính sai, kể cả cách chưa nghĩ ra
TC-A3-013 tên toàn khoảng trắng	BUG-A3-06	Cần biết <code>" "</code> là truthy trong JavaScript — kiến thức về cài đặt , không có trong đặc tả

Kết luận thiết kế: **G6b là bắt buộc, không phải tùy chọn**. Bộ sinh được thiết kế để làm người kiểm thử rảnh tay khỏi phần cơ học, chứ không phải để thay thế họ.

5. Bài học từ chính lần chạy này (đã đưa vào thiết kế)

Hai khiếm khuyết được phát hiện trong **chính bộ test**, không phải trong SUT. Cả hai đều xác nhận nguyên tắc N1 (khuôn mã dùng chung):

5.1 — Assertion bất đồng bộ nuốt mất lỗi.

Khuôn mã ban đầu sinh ra:

```
pm.test('...', function (done) {
  pm.sendRequest(opts, function (err, res) {
    pm.expect(res.json().role).to.eql('user');    // ném lỗi -> done() không bao giờ chạy
    done();
  });
});
```

Khi assertion **đạt**, `done()` chạy và Newman ghi nhận PASS. Khi assertion **hỏng**, ngoại lệ được ném trước `done()`, và Newman **âm thầm bỏ qua test đó** — không PASS, không FAIL, biến mất khỏi báo cáo. Hậu quả: BUG-A1-01 (leo quyền lên admin, lỗi nghiêm trọng nhất tìm được) ban đầu hiện ra như "đã pass".

Đã sửa trong khuôn: mọi assertion bất đồng bộ được bọc `try { ... done(); } catch (e) { done(e); }`. Sửa một chỗ trong bộ biên dịch → 32 assertion được sửa cùng lúc.

Đưa vào thiết kế: bộ kiểm tra G6a nay chặn mọi descriptor `exec` có `pm.sendRequest` mà không có `catch (e) { done(e); }`.

5.2 — Trạng thái chuẩn bị đặt trong pre-request script là không đáng tin.

Việc chụp mốc đối chiếu (`countBefore`) bằng `pm.sendRequest` trong pre-request script không đảm bảo hoàn tất trước khi request chính được gửi, khiến giá trị mốc bị lệch hoặc `undefined`.

Đã sửa: thay bằng 26 request `[SETUP]` tường minh — tuần tự, quan sát được trong báo cáo, và tự nó cũng có assertion.

Đưa vào thiết kế: G2–G5 sinh **request setup tường minh** cho mọi điều kiện tiên quyết, không dùng tác dụng phụ trong pre-request script.

6. Đánh giá thiết kế trên chính bài tập này

Chỉ số	Kết quả
Test case sinh ra	144 (110 do AI, 34 do con người thêm)
Phủ kỹ thuật	84 phân vùng miền · 18 chuyển trạng thái · 28 bảo mật · 14 lược đồ
Lỗi thật tìm được	24 (4 Critical, 7 High, 8 Medium, 5 Low)
Lỗi chỉ tìm được nhờ case con người	6 / 24
Case AI phải hiệu chỉnh khi audit	9 / 110 (8,2 %)
Khiếm khuyết của chính bộ test bị phát hiện	2 (đều nằm ở khuôn mã dùng chung)

7. Tài liệu liên quan

Nội dung	Vị trí
Pseudocode của thiết kế	<code>pseudocode/generator_pseudocode.md</code>
Cài đặt tham chiếu (chạy được)	<code>pseudocode/generator.py</code>
Agent Skill tái sử dụng	<code>SKILL.md</code>
Đặc tả sơ đồ cần tự vẽ	<code>diagram/README.md</code>
Bộ biên dịch IR → Postman (đang dùng thật)	<code>../postman/scripts/build_collections.py</code>